TECHNICAL REPORT · IMPLEMENTATION NOTE

Open-Vocabulary Tagging and Object Still Generation from Browser 3D Gaussian Splatting

Gaussian Splatting Web Viewers — WebGPU path

2 September 2026 · Repository implementation note, not a claim of new reconstruction metrics.

Abstract

This report describes a modular pipeline that (i) loads a 3D Gaussian Splatting (3DGS) scene in the browser, (ii) rasterizes it with a WebGPU implementation of the Kerbl et al. image-formation model, (iii) captures calibrated 2D views, (iv) tags those views with a vision–language model, and (v) produces object stills with an image-edit API. The design follows the 2D-lift paradigm of PointGS, GALA, and LangSplat: language is applied to images that are composites of known Gaussians under known cameras, not to unconditioned text-to-image samples. Grok Imagine Image 2.0 is used only as an *edit* of 3DGS crops. NVIDIA ArtiFixer is specified as an optional later stage for repairing off-trajectory views. This document is an implementation note for the Gaussian-Splatting-WebViewers repository.

Keywords: *3D Gaussian Splatting, WebGPU, open-vocabulary tagging, Grok Imagine, GaussForge, ArtiFixer*

1. Introduction

3D Gaussian Splatting [1] represents a scene as anisotropic 3D Gaussians and rasterizes them with EWA splatting [2] and spherical-harmonic view-dependent color. Official training output is an INRIA point_cloud.ply (float covariance, SH degree 3). Compact .splat files [6] discard SH rest bands and quantize rotations.

Browser viewers historically packed everything to 32-byte rows and looked like point clouds. This project's WebGPU viewer keeps float Gaussians and SH0–3. On top of that rasterizer we add a **local sidecar** that:

1. Tags captured 3DGS frames with **Grok 4.6 vision**.
2. Clusters tags across views by a normalized name key.
3. Calls **Grok Imagine Image 2.0** to edit 3DGS crops into object cards, stored under img_output/.

The scientific constraint, taken from PointGS [10] and related open-vocab 3DGS work [11–15], is:

> *Pixels used for semantics must be produced by the Gaussian image-formation model (or a camera-conditioned repair of it). Unconditioned image generators cannot back-project a mask onto Gaussian index i.*

2. Related work — what we reuse vs. what we do not ship

Table 1 distinguishes methods whose *equations or formats* are in the running system from methods whose *ideas* informed the pipeline but whose code is not vendored.

Line of work	Role in this pipeline
Kerbl et al. 3DGS [1]; diff-gaussian-rasterization [3]	Image formation, SH eval, 3-σ Gaussian, $\alpha = o \exp(-½ r^2)$
Zwicker EWA [2]	Screen-space covariance $J W \Sigma W^T J^T$
PointGS [10]	Render 3DGS then tag 2D. Idea only — no CUDA training copied
GALA [11], LangSplat [12], Feature3DGS [13], OpenGaussian [14], Gaussian Grouping [15]	Open-vocab / instance fields on 3DGS. Not vendored
ReferSplat [16]	Referring segmentation (sentence → 3D mask). Different product; not implemented

Line of work	Role in this pipeline
NVIDIA ArtiFixer [17, 18]	Optional camera+opacity video repair. NVIDIA noncommercial weights; not in the MIT tree
NVIDIA Fixer / Difix3D+ [19]	Distinct single-step *image* model — do not confuse with ArtiFixer
GaussForge [4]	Multi-format decode IR (PLY, SPLAT, SPZ, KSPLAT, SOG)
antimatter15/splat [6], Kellogg [7], quadjr [8]	Heritage WebGL viewers and 32-byte layout
xAI Grok vision + Imagine Image 2.0 [20, 21]	Tagging and object-card *edits*

Table 1. Related work and its role. Code copied or linked is separated from ideas only in §7.

3. System overview

Default scene: `splats/model.splat` (compact 32-byte SH0, ~401k Gaussians). Serve the **repository root** on port 8090 so the worker can import `../shared/splat-io.js`. The sidecar on 8766 reads `XAI_API_KEY` from the repo-root `.env`. The browser never sees API keys.

3.1 Flowchart

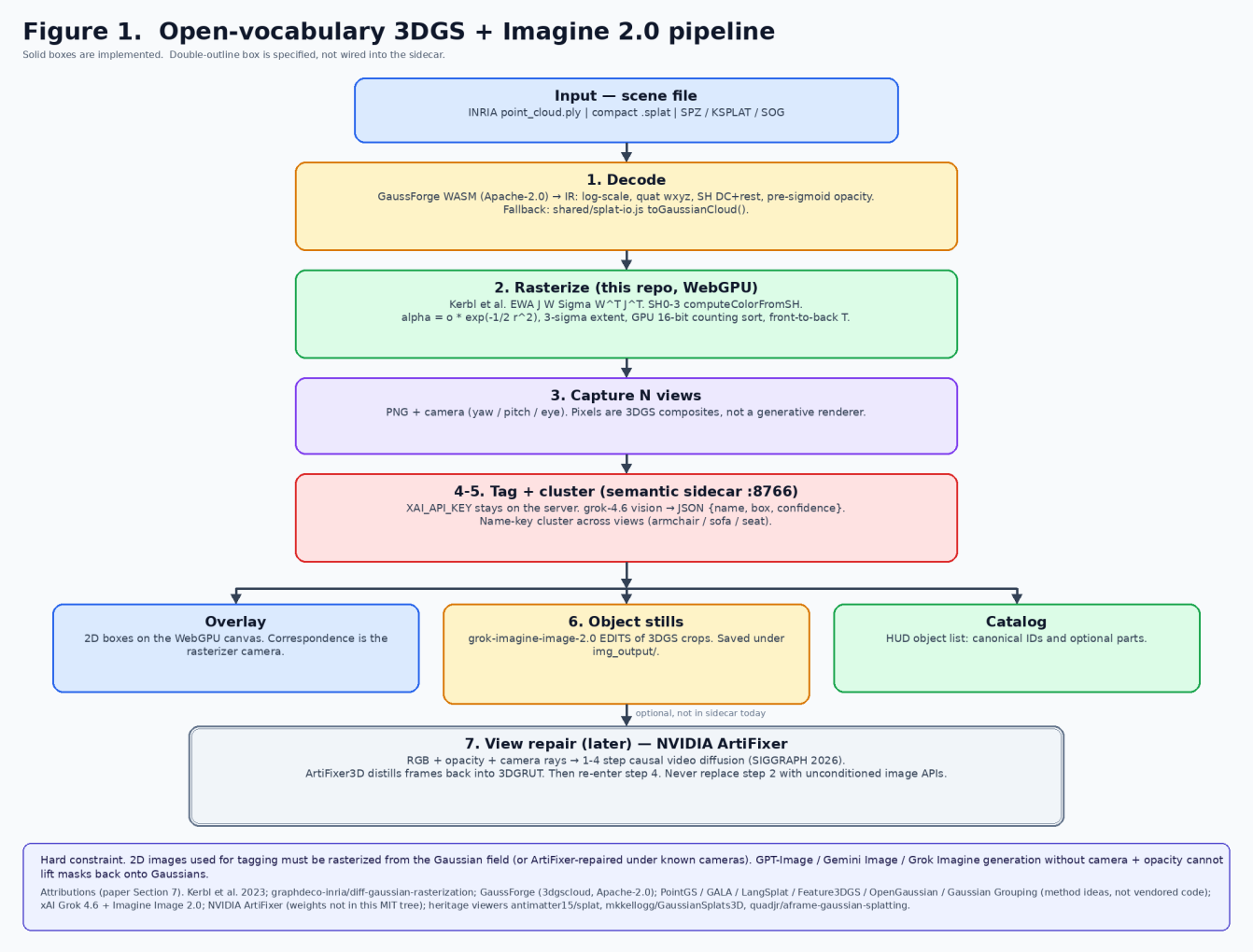


Figure 1. End-to-end workflow. Solid path is implemented. ArtiFixer (double outline) is specified but not wired into the sidecar.

Stage	Input	Output	Engine	Code
1 Decode	Bytes + filename	gaussians N×12, sh N×48	GaussForge; fallback toGaussianCloud	parse-worker.js, shared/splat-io.js
2 Rasterize	Float cloud	Premultiplied framebuffer	WebGPU, Kerbl image formation	gpu-renderer.js
3 Capture	Current / orbit cameras	PNG + yaw/pitch/eye	canvas snapshotPng	main.js
4 Tag	PNG	{name, box, confidence, parts}	grok-4.6 vision	semantic_sidecar/server.py
5 Cluster	Per-view tags	Canonical objects	Normalized name key	same
6 Cards	Crop of best_box	Studio still	grok-imagine-image-2.0 **edits**	same; files in img_output/
7 Repair (later)	RGB, opacity, rays	Repaired video / 3DGRUT	NVIDIA ArtiFixer	not in sidecar

Table 2. Pipeline stages, I/O, engines, and source files.

4. Rasterizer (stage 2)

The WebGPU path does **not** pack to 32-byte rows for drawing. Each Gaussian is position, opacity, scale, quaternion (wxyz), plus 16 RGB spherical-harmonic coefficients.

Following Kerbl et al. [1] and the CUDA reference [3]:

- Covariance $\Sigma = R\,S\,S^T\,R^T$.
- EWA 2D covariance from the Jacobian J of the projective map and the view rotation W.
- Color from `computeColorFromSH` (SH_C0 … SH_C3), then +0.5, max(0, ·).
- Extent $3\sqrt{\lambda}$ (paper 3-sigma).
- Fragment $\alpha = \min(0.99, o \cdot \exp(-\tfrac{1}{2} r^2))$.
- Front-to-back transmittance with blend `(oneMinusDstAlpha, one)` and clear alpha 0.

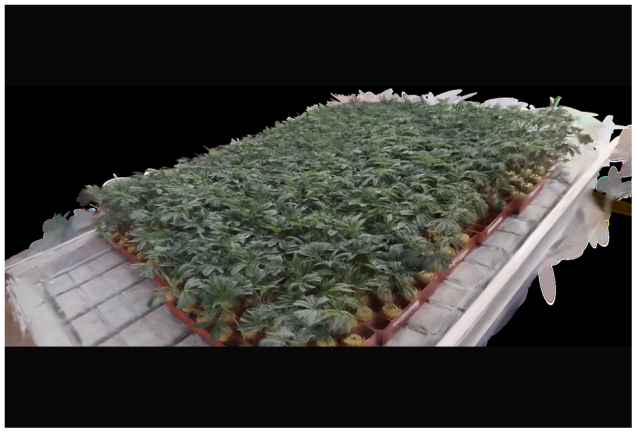
Compact `.splat` still only recovers SH0; the HUD warns. A full radiance field requires INRIA `point_cloud/iteration_*/point_cloud.ply` with `f_rest_0…44`. Counting sort is a 16-bit GPU histogram + prefix sum + scatter (near first). It is not copied from a named third-party WebGPU 3DGS engine.

5. Semantic sidecar (stages 4–6)

The browser POSTs PNG captures to `http://127.0.0.1:8766`. Keys never leave the sidecar process.

- **Vision.** `POST https://api.x.ai/v1/chat/completions` model `grok-4.6`, image + JSON-schema prompt. Fallback `POST /v1/responses`.
- **Cluster.** Lowercase alphanumeric key; merge “armchair / green sofa / seat” by string identity (not CLIP [9]; a later upgrade).
- **Imagine.** `POST https://api.x.ai/v1/images/edits` model `grok-imagine-image-2.0`, `response_format: b64_json`. The prompt asks for a photoreal product still of the **reference crop**. This is image-to-image, not text-to-image from nothing.
- **Library.** `img_output/YYYYMMDD-HHMMSS-name/{source.png, imagine.jpg, meta.json}` plus `img_output/index.jsonl`.

A demo crop of potted plants on a nursery table (Figure 2) was rasterized in the WebGPU viewer and edited with Imagine 2.0. Files: `img_output/20260902-164810-potted-plants/`.



(a) 3DGS crop — potted plants on a nursery table

(b) Imagine 2.0 edit of (a) — object still

Figure 2. (a) WebGPU 3DGS crop of potted plants on a nursery table. (b) Grok Imagine Image 2.0 edit of that crop. The still is an illustration of the tagged object, not a measurement of the Gaussian field.

6. What must not be done

1. Replace stage 2 with GPT-Image, Gemini Image, or Imagine **generation**. Those APIs have no camera or opacity, so correspondence to Gaussians is lost [10].
2. Feed Imagine outputs back into SAM / VLM distillation as if they were 3DGS rasters.
3. Put `XAI_API_KEY` in frontend JavaScript.
4. Vendor ArtiFixer 14B weights into this MIT repository (NVIDIA OneWay Noncommercial license [18]).
5. Confuse ArtiFixer [17] with nvidia/Fixer (Difix3D+) [19].

7. Attributions and licenses

This repository's WebGL heritage and the WebGPU / semantic additions use third-party methods and software as follows. **Code copied or linked** is distinguished from **ideas only**.

7.1 Software used in the running system

Component	Origin	License	How we use it
3DGS image formation, SH basis, 3-σ Gaussian	Kerbl et al. [1]; CUDA reference [3] (graphdeco-inria/diff-gaussian-rasterization)	INRIA research license on the CUDA lib; equations reimplemented in WGSL	gpu-renderer.js
Multi-format Gaussian IR	GaussForge [4] @gaussforge/wasm	Apache-2.0	parse-worker.js (jsDelivr)
PLY / splat fallback parser	this repo shared/splat-io.js, layout after [6]	MIT (this repo)	Worker fallback
Compact 32-byte .splat	antimatter15/splat [6] (Kevin Kwok)	MIT	Format interop; not the WebGPU GPU buffer
Viewer 1	quadjr/aframe-gaussian-splatting [8]	MIT	gaussian_splatting_1/
Viewer 2	mkkellogg/GaussianSplats3D [7]	MIT	gaussian_splatting_2_*
Grok 4.6 vision, Imagine Image 2.0	xAI API [20, 21]	xAI API terms	Sidecar only
Repo license	Akbar S. / forks	MIT	LICENSE

Table 3. Software in the running system.

7.2 Methods that informed the design (no code vendored)

Method	Taken from it	Not taken
PointGS [10] — Song et al., CVPR 2026, arXiv:2605.11520	Render 3DGS then tag 2D; do not tag raw point projections	CUDA 3DGS training, SAM contrastive distillation, 2-step ICP
GALA [11] — Alegret et al., arXiv:2508.14278	Open-vocab 2D+3D on 3DGS; codebook / instance consistency as a goal	Scaffold-GS training, dual codebooks
LangSplat [12] — Qin et al., CVPR 2024	Distill language from 2D views of 3DGS	Autoencoder CLIP fields per Gaussian
Feature3DGS [13] — Zhou et al., CVPR 2024	Feature rasterization idea	CNN feature lifting / parallel N-D rasterizer
OpenGaussian [14] — Wu et al., NeurIPS 2024	Instance clustering + language	Hierarchical CUDA clustering
Gaussian Grouping [15] — Ye et al., ECCV 2024	Lift 2D masks toward 3D instances	Editing GUI
ReferSplat [16] — He et al., ICML 2025	— (different task: referring 3DGS segmentation)	Referring field training, Ref-LERF
ArtiFixer [17, 18] — de Lutio et al., SIGGRAPH 2026	Opacity-mixing, camera-conditioned repair, 1–4 step AR video; 3DGRUT distill	Weights, Docker training
SPZ / Niantic GaussianCloud [5]	IR field meanings (log scale, pre-sigmoid alpha, SH layout)	Compression codec (via GaussForge)

Table 4. Design sources that are not vendored into this tree.

7.3 License compatibility (summary)

- **This MIT tree** may include GaussForge (Apache-2.0), antimatter15 / Kellogg / quadjr (MIT), and a WGSL reimplementation of published 3DGS equations.
- **xAI** is a runtime service: no model weights in git.
- **ArtiFixer weights** must stay out of this repo (NVIDIA OneWay Noncommercial). Code on GitHub is Apache-2.0 but the checkpoint is not.
- **INRIA** `diff-gaussian-rasterization` is not copied; only the published forward formulas [1, 3] are reimplemented.

8. How to run the implemented path

```
cd Gaussian-Splatting-WebViewers
python3 -m http.server 8090 --bind 127.0.0.1
./semantic_sidecar/launch.sh # :8766
./gaussian_splatting_webgpu/launch-webgpu-chrome.sh
```

Default URL loads `../splats/model.splat`. In the HUD: **Tag scene (Grok)** with **Imagine 2.0 object cards** checked. Cards appear on the right and on disk under `img_output/`. For SH3 radiance fields, drop a trained `point_cloud.ply` instead of the compact splat. Operator notes: `docs/pipeline.md`. Chrome / Vulkan: `docs/webgpu-chrome.md`.

9. Limitations

- Compact `.splat` is SH0; Imagine stills can look sharper than the 3DGS source because the edit model hallucinates high-frequency texture. They are illustrations, not measurements.
- Clustering is string-based, not CLIP embeddings [9].
- No SAM 2, no contrastive Gaussian affinity field, no ArtiFixer GPU in this process.
- Vision may under-label blurry splat views; zoom before tagging.
- Imagine URL downloads can 403; the sidecar requests `b64_json`.

Appendix A. File map

Path	Role
gaussian_splatting_webgpu/	Viewer, rasterizer, capture, HUD

Path	Role
semantic_sidecar/server.py	Vision + Imagine + img_output/
shared/splat-io.js	Shared I/O (PLY / splat fallback)
img_output/	Imagine library (gitignored except README)
docs/pipeline.md	Short operator notes
docs/webgpu-chrome.md	Chrome / Vulkan launch
docs/figures/pipeline-flowchart.png	Figure 1
docs/open-vocab-3dgs-imagine-pipeline-paper.md	This report (Markdown)
docs/open-vocab-3dgs-imagine-pipeline-paper.pdf	This report (PDF)

Table 5. Paths mentioned in this report.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” ACM Trans. Graph., vol. 42, no. 4, 2023. https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/ Code: https://github.com/graphdeco-inria/gaussian-splatting

[2] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, “EWA Splatting,” IEEE Trans. Visualization and Computer Graphics, 2002.

[3] GRAPHDECO / Inria, diff-gaussian-rasterization (forward.cu: computeColorFromSH, computeCov2D, computeCov3D). https://github.com/graphdeco-inria/diff-gaussian-rasterization

[4] 3dgscloud, GaussForge, Apache-2.0. https://github.com/3dgscloud/GaussForge WASM: @gaussforge/wasm

[5] Niantic, SPZ / Gaussian cloud IR. https://github.com/nianticlabs/spz

[6] K. Kwok, antimatter15/splat, MIT. https://github.com/antimatter15/splat

[7] M. Kellogg, GaussianSplats3D, MIT. https://github.com/mkkellogg/GaussianSplats3D

[8] K. Kwok and J. Kuwada, quadjr/aframe-gaussian-splatting, MIT. https://github.com/quadjr/aframe-gaussian-splatting

[9] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision” (CLIP), ICML 2021.

[10] Y. Song, Q. Li, W. Wang, and Z. Yan, “PointGS: Semantic-Consistent Unsupervised 3D Point Cloud Segmentation with 3D Gaussian Splatting,” CVPR 2026. arXiv:2605.11520. https://github.com/SebastianYIXIAO/pointGS

[11] E. Alegret, K. Li, S. Wang, S. Liang, M. Niemeyer, S. Gasperini, N. Navab, and F. Tombari, “GALA: Guided Attention with Language Alignment for Open Vocabulary Gaussian Splatting,” arXiv:2508.14278, 2025.

[12] M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister, “LangSplat: 3D Language Gaussian Splatting,” CVPR 2024. arXiv:2312.16084.

[13] S. Zhou, H. Chang, S. Jiang, Z. Fan, Z. Zhu, D. Xu, P. Chari, S. You, Z. Wang, and A. Kadambi, “Feature 3DGS: Supercharging 3D Gaussian Splatting to Enable Distilled Feature Fields,” CVPR 2024. arXiv:2312.03203.

[14] Y. Wu, J. Meng, H. Li, C. Wu, Y. Shi, X. Cheng, C. Zhao, H. Feng, E. Ding, J. Wang, and J. Zhang, “OpenGaussian: Towards Point-Level 3D Gaussian-based Open Vocabulary Understanding,” NeurIPS 2024. arXiv:2406.02058.

[15] M. Ye, M. Danelljan, F. Yu, and L. Ke, “Gaussian Grouping: Segment and Edit Anything in 3D Scenes,” ECCV 2024. arXiv:2312.00732.

[16] S. He, G. Jie, C. Wang, Y. Zhou, S. Hu, G. Li, and H. Ding, “ReferSplat: Referring Segmentation in 3D Gaussian Splatting,” ICML 2025. arXiv:2508.08252. https://github.com/heshuting555/ReferSplat

[17] R. de Lutio, T. Fischer, Y.-Y. Chang, Y. Zhang, J. Z. Wu, X. Ren, T. Shen, K. Tothova, Z. Gojcic, and H. Turki, “ArtiFixer: Enhancing and Extending 3D Reconstruction with Auto-Regressive Diffusion Models,” SIGGRAPH 2026. https://research.nvidia.com/labs/sil/projects/artifixer/

[18] NVIDIA, nvidia/ArtiFixer (weights: NVIDIA OneWay Noncommercial License; Wan 2.1 base: Apache-2.0). https://huggingface.co/nvidia/ArtiFixer Code: https://github.com/nv-tlabs/artifixer

[19] J. Wu et al., Difix3D+ / NVIDIA Fixer, arXiv:2503.01774. https://huggingface.co/nvidia/Fixer — not ArtiFixer.

[20] xAI, Grok image understanding (grok-4.6). https://docs.x.ai/docs/guides/image-understanding

[21] xAI, Imagine Image 2.0 (grok-imagine-image-2.0), generations and edits. https://docs.x.ai/developers/model-capabilities/images/generation https://docs.x.ai/developers/model-capabilities/images/editing

[22] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik, “LERF: Language Embedded Radiance Fields,” ICCV 2023.

[23] A. Kirillov et al., “Segment Anything,” ICCV 2023. (PointGS / GALA mask source; not in our sidecar.)